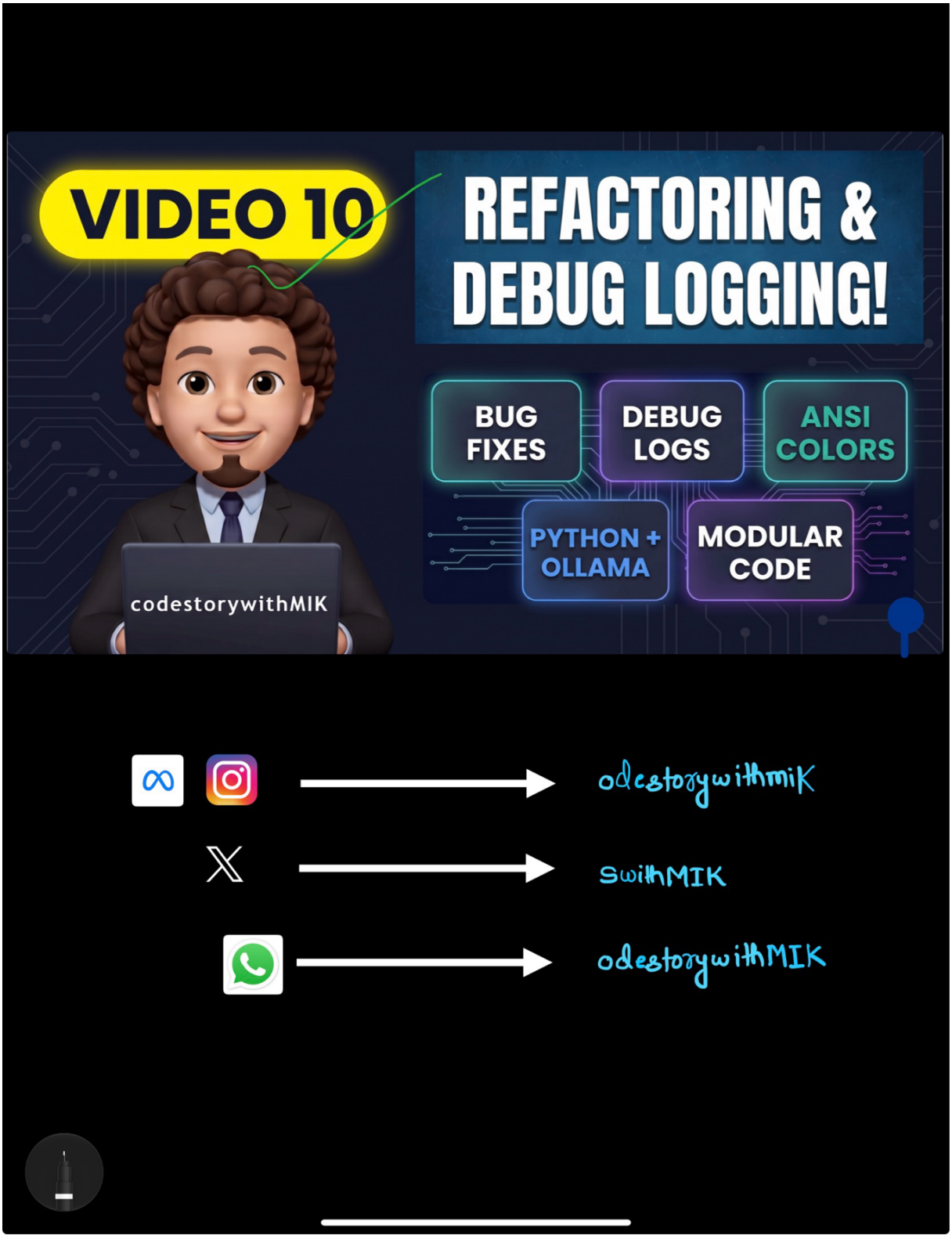
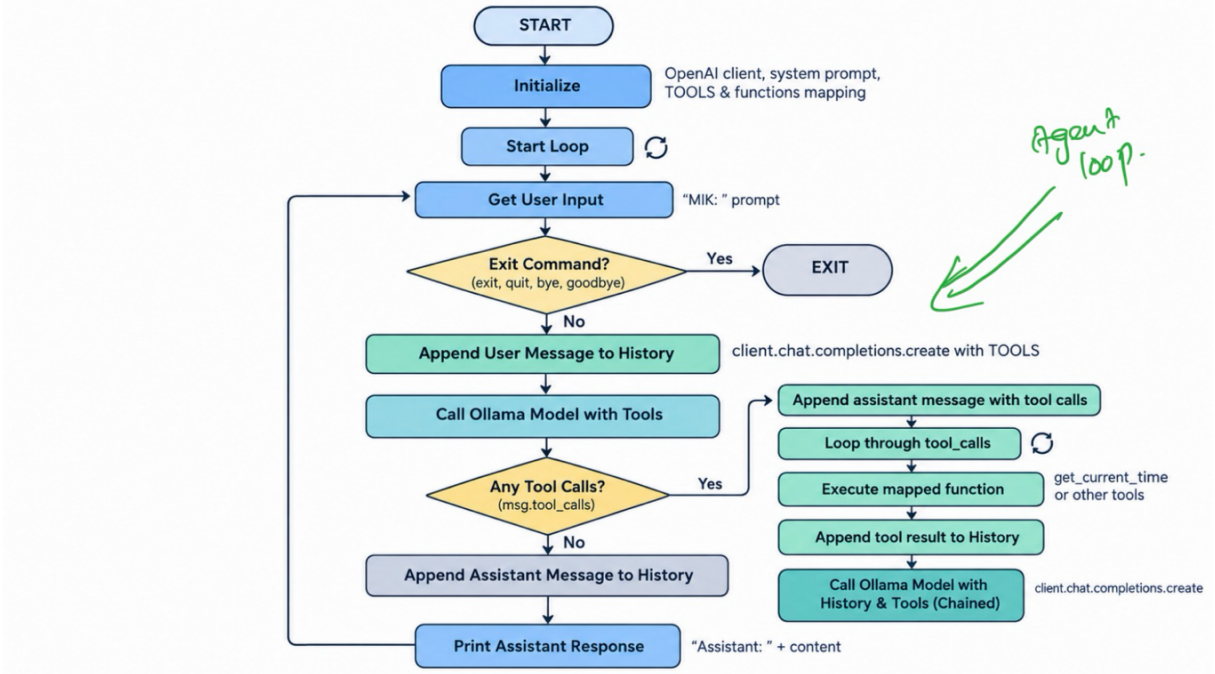

VIDEO 10
REFACTORING & DEBUG LOGGING!
BUG FIXES
DEBUG LOGS
ANSI COLORS
PYTHON + OLLAMA
MODULAR CODE
codestorywithMIK
codestorywithmiK
SwithMIK
codestorywithMIK

Recap : video - 9



(•) We added MAX_STEPS → Safe loop for tool calling

(•) try Exception handling for tool crashes.

Let's Refactor and Fix some small bugs



```python
def get_city_weather(city):
    url = f"https://wttr.in/{city}?format=3"
    response = requests.get(url, timeout=10)
    return response.text
```

```
{
    "type" : "function",
    "function" : {
        "name" : "get_city_weather",
        "description" : "Returns the current weather for a given city.",
        "parameters" : {
            "type" : "object",
            "properties" : {
                "city" : {
                    "type" : "string",
                    "description" : "City to get the weather for.",
                }
            },
        },
    },
},
```



the weather ?

User : What is the weather ?

The model may call get_city_weather with empty arguments.

Our code crashes — Missing Argument !!!

Note: (•) whatever tool description we give to model. is a Hint to model not a guarantee.

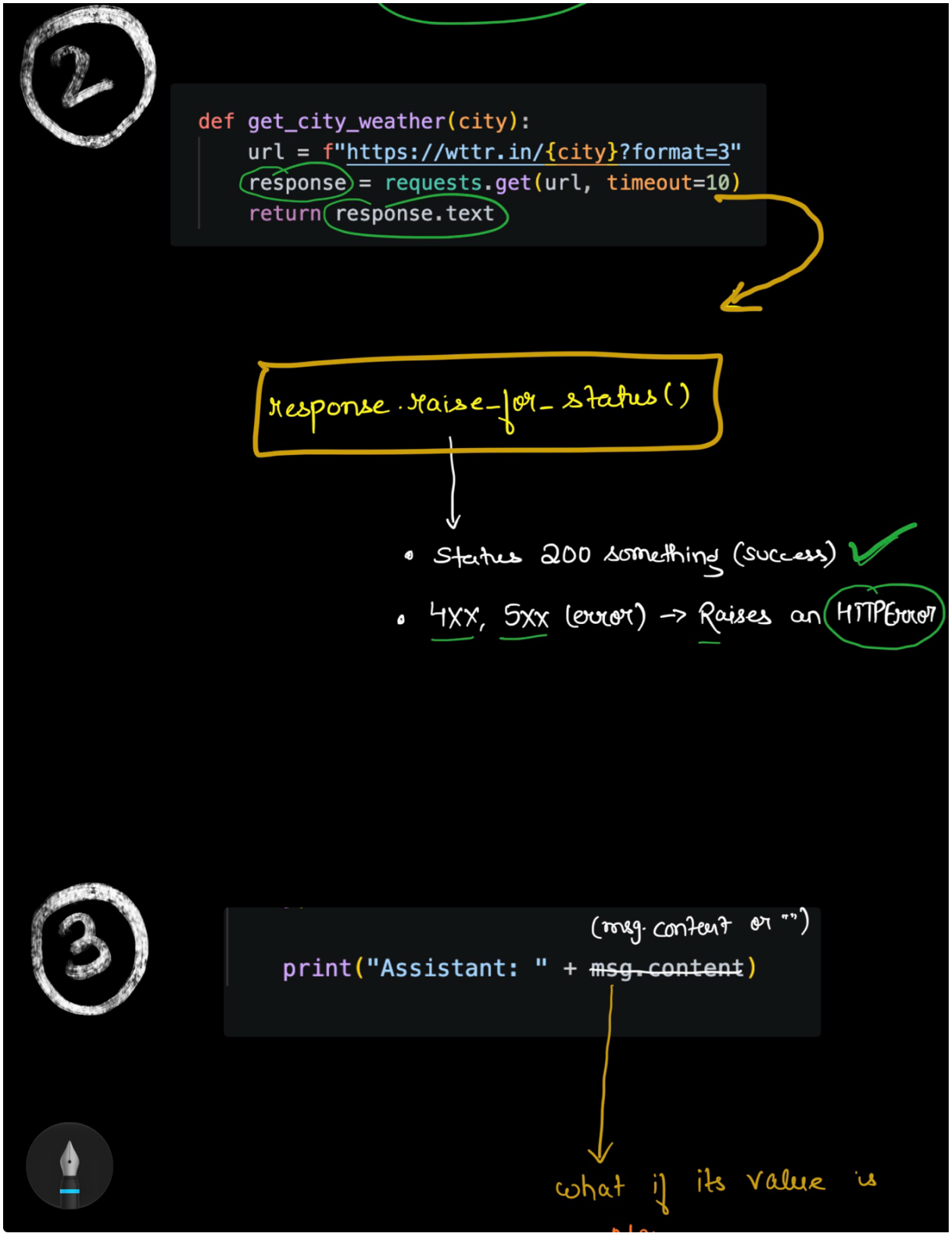
(•) Small models may still ignore instructions.

(•) So, make it a habit to not rely on model totally.

↳ That's why we added Exception handling

↳ We can also handle API call gracefully.




2
def get_city_weather(city):
url = f"https://wttr.in/{city}?format=3"
response = requests.get(url, timeout=10)
return response.text
response.raise_for_status()
• Status 200 something (success)
• 4XX, 5XX (error) -> Raises an HTTPError
3
(msg.content or "")
print("Assistant: " + msg.content)
what if its value is

How to access it ?

```
if os.getenv("DEBUG") == "True":
    //print debug logs
```

My Personal preference on printing debug logs on terminal

- It should be clear & concise
- Short, meaningful
- Different color
- One method which recieves the debug log and it's only responsibility is to print them.



Importance of Debugging



Whenever I want to debug my code it should be doable with minimal changes

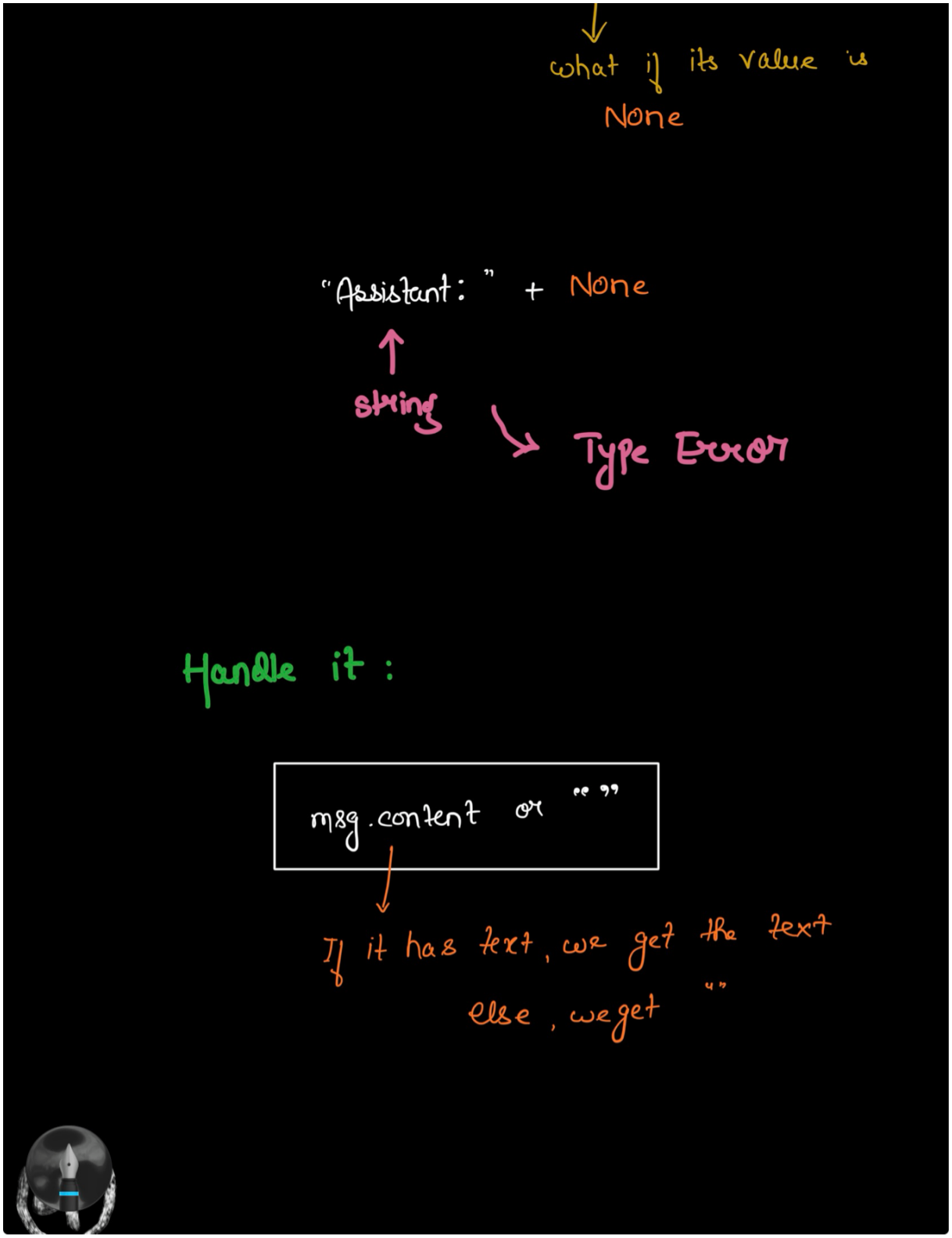
- Add a DEBUG mode to your agent.
- when DEBUG = True → print debug logs
  DEBUG = False → Run normally

- I will store it in .env file

```
.env
1  BASE_URL=http://127.0.0.1:11434/v1
2  API_KEY=dummy_key
3  MODEL=qwen3:4b
4  DEBUG=True
```



"True"


what if its value is None
"Assistant: " + None
string
Type Error
Handle it :
msg.content or ""
If it has text, we get the text
else, we get ""

\033 → It tells the terminal that next characters are command

[→ start of command

90 → Command : set text color to grey.

m → End of command

Example:- print ("Hello") → "Hello"

print ("\033[91m Hello") → "Hello"

(\033 : signal)

Note:- The command should end as well. Else all texts after it



Utilities / utilities.py

```
def log(text):
    if(os.getenv("DEBUG") == "True":
        print(f"  {text}")
```

Coloring the Debug logs

ANSI Escape Codes

(only for fun. You can skip it)

- Special characters
- Terminals / Emulators read them as commands.

Example: \033 , \x1b etc.

```
print("\033  Hello")
```



- Use a standard logging library (we will only do print() for simplicity).
- Put logs where we have chances of failure.

1. log what & how many tools model requested.
2. log tool name | arguments
3. log result from tool
4. Keep such utilities in a separate folder Utilities/utilities.py

Example:

```
(.venv) ┌─MIK ~/Desktop/Simple AI Assistant
└─→ python3 simple_ai_assistant.py

MIK: Hi, tell me the current time and also tell me the weather of Seattle
  Step 1: model requested 2 tool call(s)
  🔧 get_current_time({})
     ↳ 2026-10-19 15:58:26
  🔧 get_city_weather({'city': 'Seattle'})
     ↳ Seattle: ☁️ +10°C

Assistant: Time: 2026-10-19 15:58. Seattle: 10°C cloudy.

MIK:
```



```
result = run_tool(tool_call)

messages.append({
    "role": "tool"
    "tool_call_id": tool_call.id,
    "content": result,
})
```

tools.py

```
def run_tool(tool_call):
    ...
    //run tool
    return str(result)
```

Let's Apply it ...

Modular Code

① We Kept `log()` inside Utilities/utilities.py

② Every time we are running tool → Move it

→ tools.py → tool execute logic

```python
for tool_call in msg.tool_calls:
    func_name = tool_call.function.name
    func_args = {}

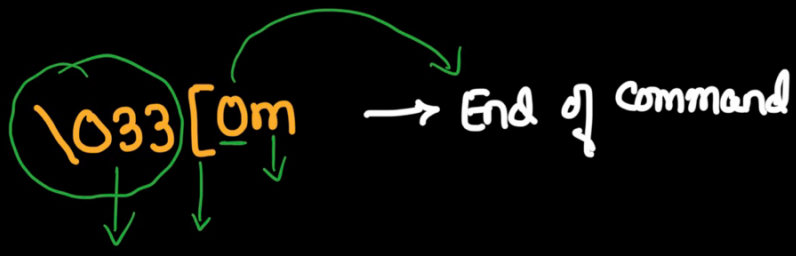
    try:
        func_args = json.loads(tool_call.function.arguments or "{}")
        log(f"🔧 {func_name}({func_args})")
        fn = FUNCTION_NAME_TO_FUNCTION_MAP[func_name]
        result = fn(**func_args)
    except Exception as e:
        result = f"Error while running {func_name} : {e}"

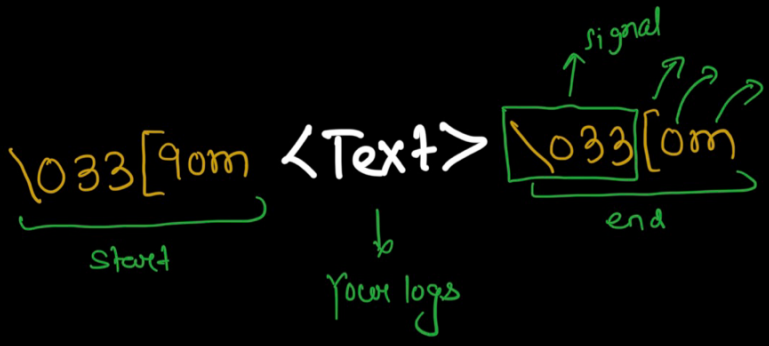
    log(f"   ↳ {result}")
    messages.append({
        "role": "tool",
        "tool_call_id" : tool_call.id,
        "content" : str(result),
    })
```

```
for tool_call in msg.tool_calls:
    result = run_tool(tool_call)
    messages.append({
```

well. Else all texts after

will be impacted.





Let's apply it